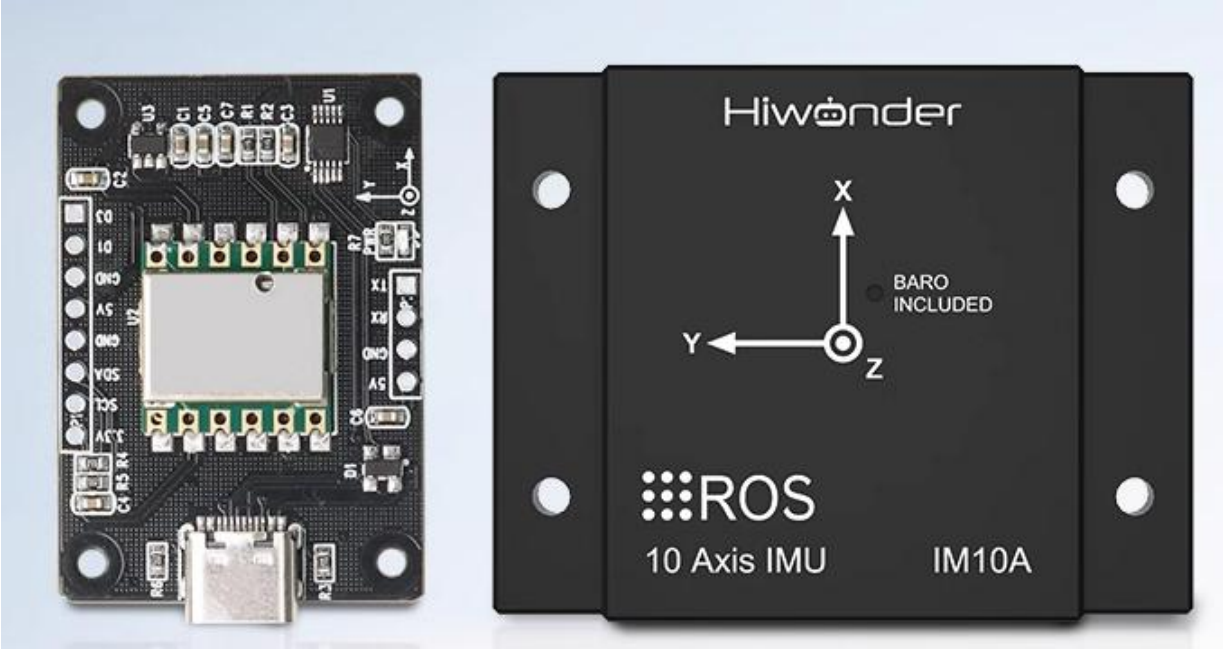
幻尔科技

IMU 应用教程-基础应用

V1.0



版本号	修改日期	修改摘要
V1.0	20231124	初次发布

目录

1. STM32-串口应用例程 3
1.1 准备工作 3
1.2 案例开发 3
1.3 实验效果 4
2. STM32-IIC 应用例程 5
1.1 准备工作 8
1.2 案例开发 9
1.3 实验效果 10
3. 树莓派-USB 应用例程 11
3.1 准备工作 11
3.1.1 硬件连接 11
3.1.2 安装串口驱动库 12
3.2 案例开发 12
3.3 实验效果 13
4. Jetson Nano-USB 应用例程 14
4.1 准备工作 14
4.1.1 准备硬件连接 14
4.1.2 安装串口驱动库 14
4.2 案例开发 14
4.3 实验效果 15

1. STM32F103-串口应用例程

本次例程使用的是 STM32F103C8T6 芯片开发板，为了演示方便以下内容简称为 STM32。

需要准备的产品：Win10/Win11 电脑、IMU 模块（数据接口已引线）、micro-USB 数据线、CH340 模块、杜邦线若干。

1.1 准备工作

将 STM32 串口 1 连接到 CH340 模块，CH340 模块插到 Win10 电脑的 USB 口。将 STM32 串口 2 连接到 IMU 模块的串口接口。

将 micro-USB 数据线连接到 STM32 和电脑的 USB 口，作为 STM32 供电。

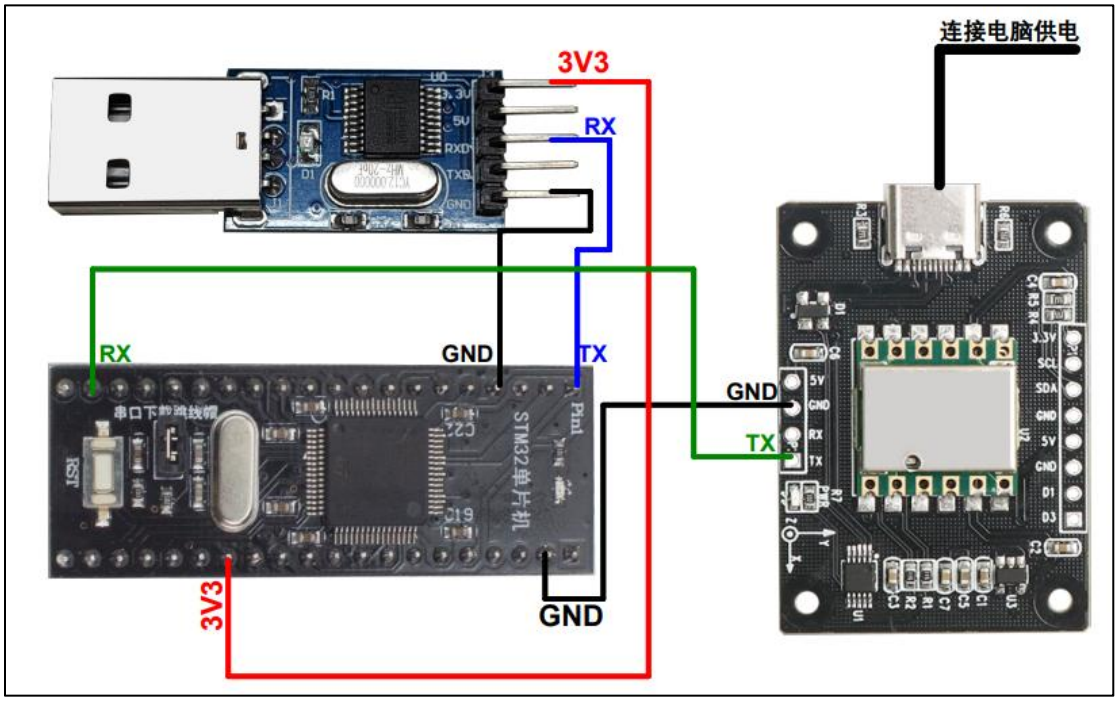
接线说明：

STM32 IMU

VCC ------> VCC

GND ------> GND

PA10 <------ TXD



1.2 案例开发

具体代码请看资料中的源码。

本例程将串口打印的波特率设置为 115200 bps，将 IMU 模块连接的波特率设置为 9600。可以根据 IMU 模块的波特率修改 STM32 串口 2 的波特率。

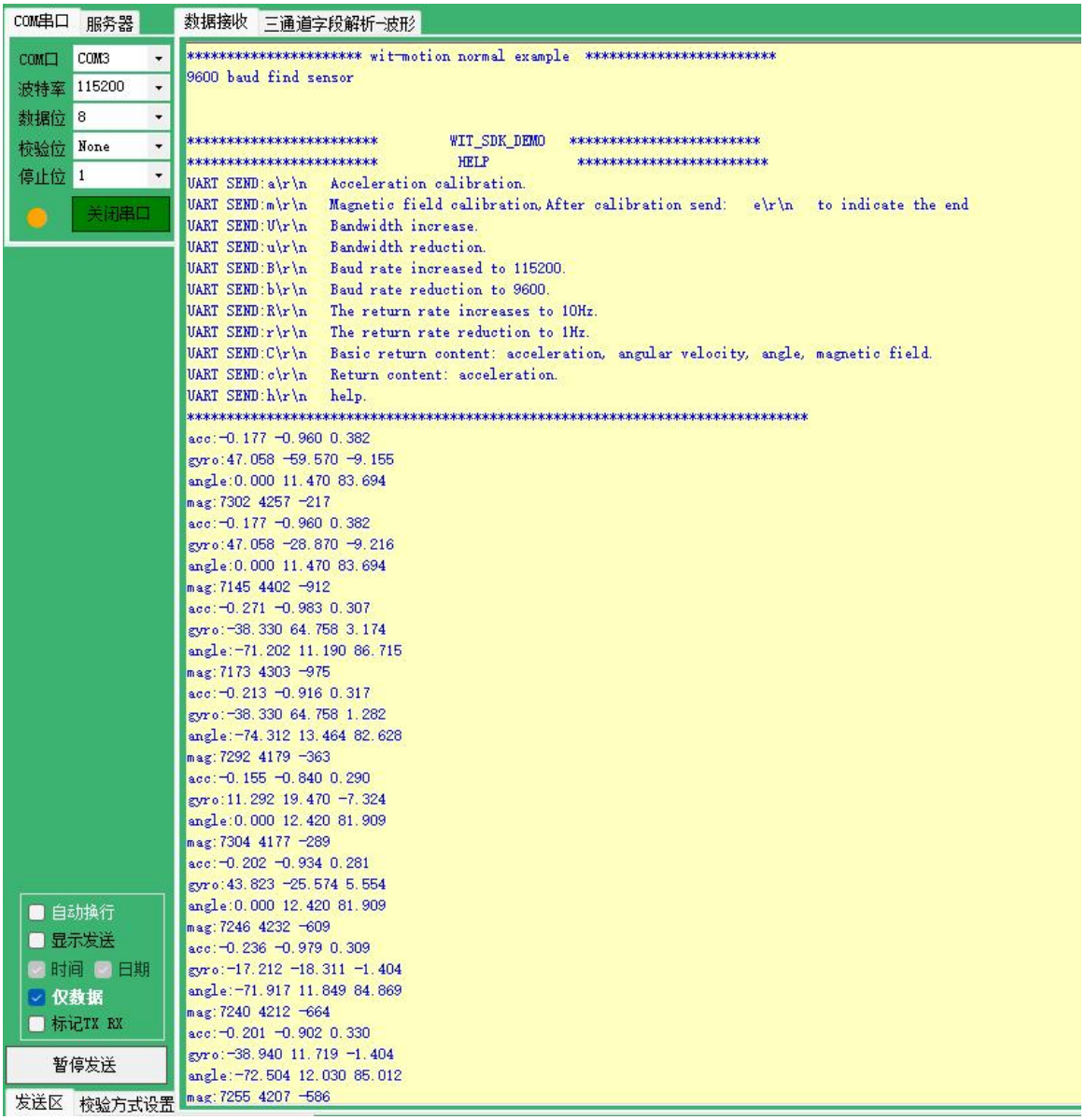
```
    Usart1Init(115200);
    Usart2Init(9600);
```

读取并打印 IMU 模块的数据。

```
59  while (1)
60  {
61    CmdProcess();
62    if(s_cDataUpdate)
63    {
64      for(i = 0; i < 3; i++)
65      {
66        fAcc[i] = sReg[AX+i] / 32768.0f * 16.0f;
67        fGyro[i] = sReg[GX+i] / 32768.0f * 2000.0f;
68      }
69      if(sReg[TEMP] - sReg[Roll] > 10 | sReg[Roll] - sReg[TEMP] > 10)
70      {
71        fAngle[0] = sReg[Roll] / 32768.0f * 180.0f;
72        fAngle_tmp = fAngle[0];
73      }
74      else   fAngle[0] = fAngle_tmp;
75
76      for(i = 1; i<3;i++)
77      {
78        fAngle[i] = sReg[Roll+i] / 32768.0f * 180.0f;
79      }
80
81      if(s_cDataUpdate & ACC_UPDATE)
82      {
83        printf("acc:%.3f %.3f %.3f\r\n", fAcc[0], fAcc[1], fAcc[2]);
84        s_cDataUpdate &= ~ACC_UPDATE;
85      }
86      if(s_cDataUpdate & GYRO_UPDATE)
87      {
88        printf("gyro:%.3f %.3f %.3f\r\n", fGyro[0], fGyro[1], fGyro[2]);
89        s_cDataUpdate &= ~GYRO_UPDATE;
90      }
91      if(s_cDataUpdate & ANGLE_UPDATE)
92      {
93        printf("angle:%.3f %.3f %.3f\r\n", fAngle[0], fAngle[1], fAngle[2]);
94        s_cDataUpdate &= ~ANGLE_UPDATE;
95      }
96      if(s_cDataUpdate & MAG_UPDATE)
97      {
98        printf("mag:%d %d %d\r\n", sReg[HX], sReg[HY], sReg[HZ]);
99        s_cDataUpdate &= ~MAG_UPDATE;
100     }
101   }
102 }
```

1.3 实验效果

程序下载进入 STM32 后，打开猫猫串口助手（配置参数如下图所示），可以看到一直打印 IMU 模块的数据，当我们改变 IMU 模块的姿态，数据会发生变化。



2. STM32F407-串口应用例程

本次例程使用的是 STM32F407C8T6 芯片开发板,为了演示方便以下内容简称为 STM32。

需要准备的产品：Win10/Win11 电脑、IMU 模块（数据接口已引线）、micro-USB 数据线、CH340 模块、杜邦线若干。

2.1 准备工作

将 STM32 串口 1 电脑的 USB 口。将 STM32 串口 2 连接到 IMU 模块的串口接口。

将 micro-USB 数据线连接到 STM32 和电脑的 USB 口，作为 STM32 供电。

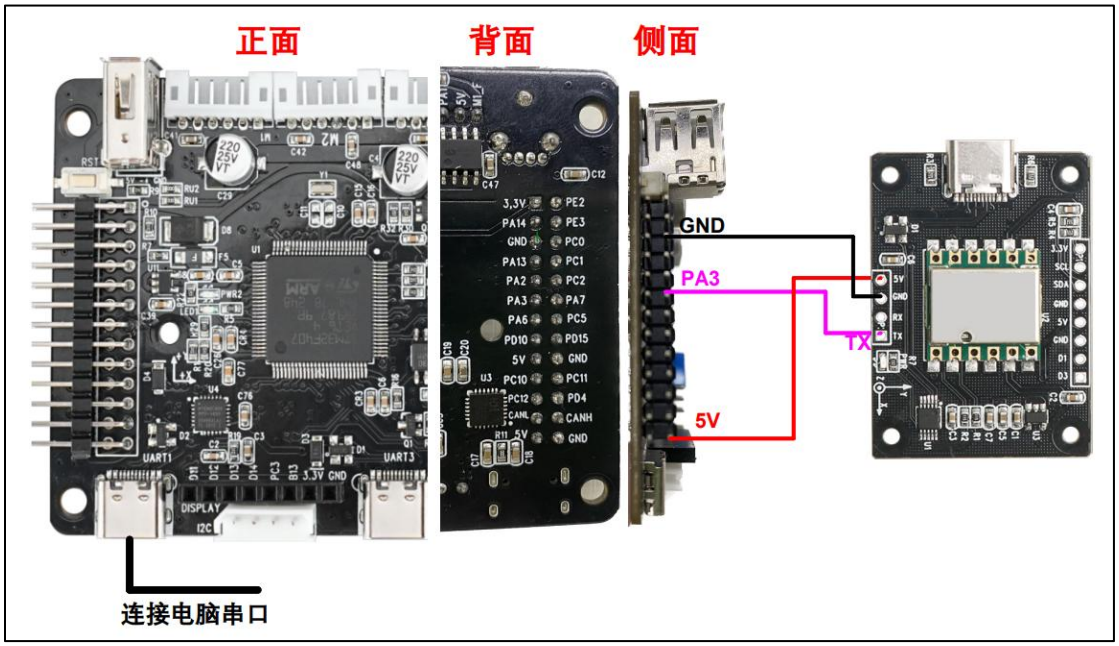
接线说明：

STM32 IMU

5V ------> 5V

GND ------> GND

PA3 <------ TXD



2.2 案例开发

具体代码请看资料中的源码。

本例程将串口打印的波特率设置为115200 bps,将IMU模块连接的波特率设置为9600。可以根据 IMU 模块的波特率修改 STM32 串口 2 的波特率。

```
MX_USART1_UART_Init();
```

```
void MX_USART1_UART_Init(void)
{

  /* USER CODE BEGIN USART1_Init 0 */

  /* USER CODE END USART1_Init 0 */

  /* USER CODE BEGIN USART1_Init 1 */

  /* USER CODE END USART1_Init 1 */
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
```

```
MX_USART2_UART_Init();
```

```
void MX_USART2_UART_Init(void)
{

  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */

  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 9600;
```

读取并打印 IMU 模块的数据。

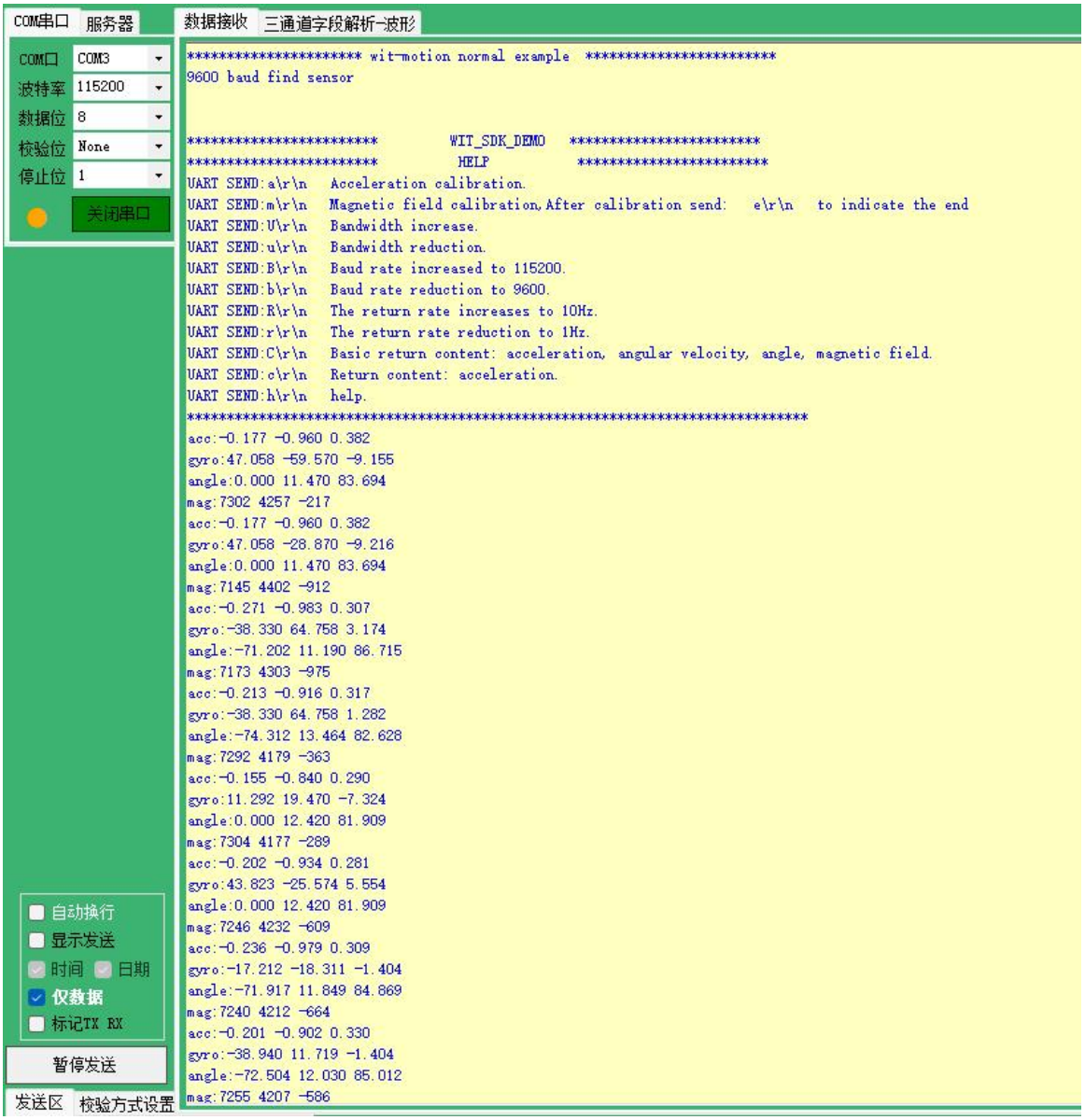
```
while (1)
{
  CmdProcess();
  if(s_cDataUpdate)
  {
      for(i = 0; i < 3; i++)
      {
          fAcc[i] = sReg[AX+i] / 32768.0f * 16.0f;
          fGyro[i] = sReg[GX+i] / 32768.0f * 2000.0f;
      }
      if(sReg[TEMP] - sReg[Roll] > 10 | sReg[Roll] - sReg[TEMP] > 10)
      {
          fAngle[0] = sReg[Roll] / 32768.0f * 180.0f;
          fAngle_tmp = fAngle[0];
      }
      else   fAngle[0] = fAngle_tmp;

      for(i = 1; i<3;i++)
      {
          fAngle[i] = sReg[Roll+i] / 32768.0f * 180.0f;
      }

      if(s_cDataUpdate & ACC_UPDATE)
      {
          printf("acc:%.3f %.3f %.3f\r\n", fAcc[0], fAcc[1], fAcc[2]);
          s_cDataUpdate &= ~ACC_UPDATE;
      }
      if(s_cDataUpdate & GYRO_UPDATE)
      {
          printf("gyro:%.3f %.3f %.3f\r\n", fGyro[0], fGyro[1], fGyro[2]);
          s_cDataUpdate &= ~GYRO_UPDATE;
      }
      if(s_cDataUpdate & ANGLE_UPDATE)
      {
          printf("angle:%.3f %.3f %.3f\r\n", fAngle[0], fAngle[1], fAngle[2]);
          s_cDataUpdate &= ~ANGLE_UPDATE;
      }
      if(s_cDataUpdate & MAG_UPDATE)
      {
          printf("mag:%d %d %d\r\n", sReg[HX], sReg[HY], sReg[HZ]);
          s_cDataUpdate &= ~MAG_UPDATE;
      }
  }
```

2.3 实验效果

程序下载进入 STM32 后，打开猫猫串口助手（配置参数如下图所示），可以看到一直打印 IMU 模块的数据，当我们改变 IMU 模块的姿态，数据会发生变化。



3. STM32-IIC 应用例程

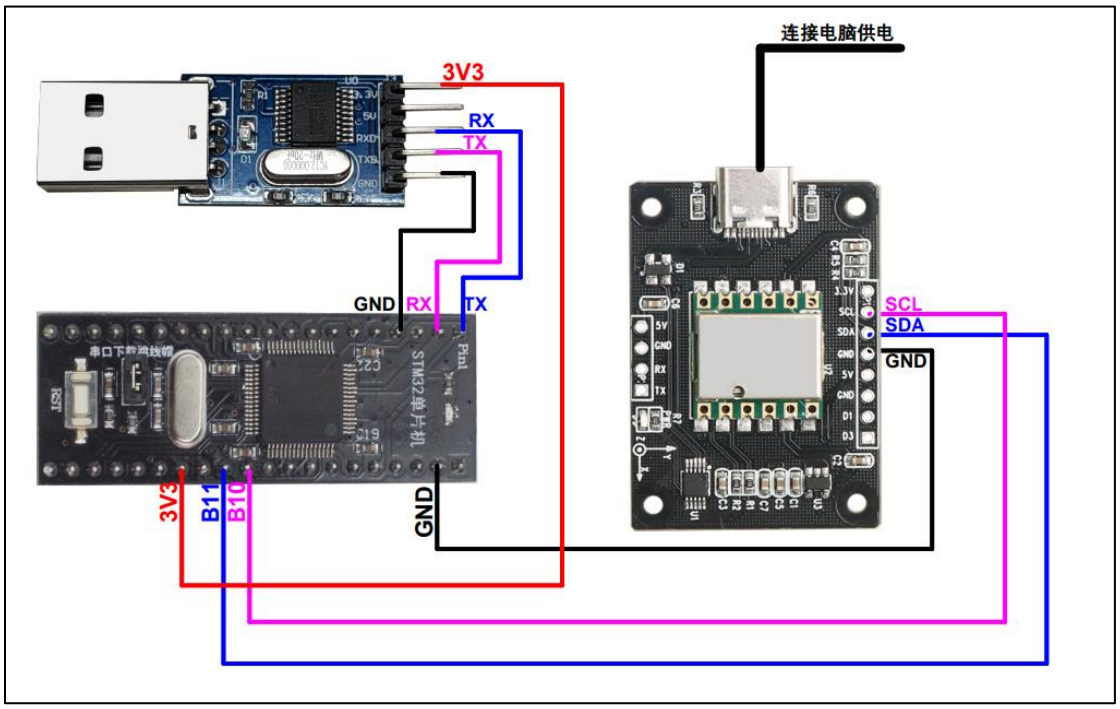
本次例程使用的是 STM32F103C8T6 芯片开发板，为了演示方便以下内容简称为 STM32。

需要准备的产品：Win10/Win11 电脑、IMU 模块（数据接口已引线）、micro-USB 数据线、CH340 模块、杜邦线若干。

3.1 准备工作

将 STM32 串口 1 连接到 CH340 模块，CH340 模块插到 Win10 电脑的 USB 口。将 STM32 的 IIC 接口 2 连接到 IMU 模块的 IIC 接口。

将 micro-USB 数据线连接到 STM32 和电脑的 USB 口，作为 STM32 供电。



3.2 案例开发

具体代码请看资料中的源码。

本例本例程已经将波特率设置为 115200 bps，注意此波特率是 STM32 串口打印的波特率，并非 IMU 模块的波特率。

```
Usart1Init(115200);
```

接下来需要注意的是 IMU 模块的 IIC 地址，默认为 0x50，如果修改了其他的地址请修改此参数。

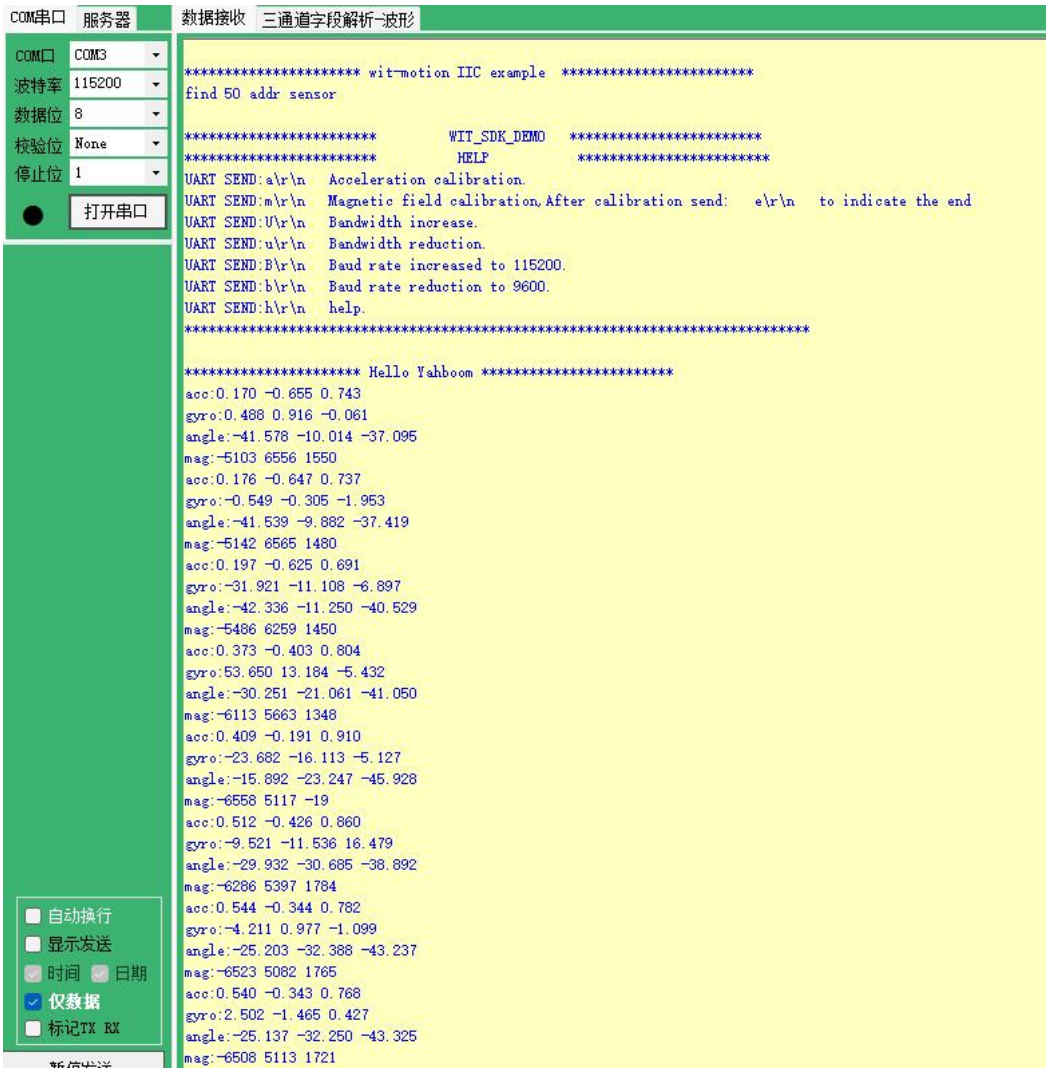
```
IIC_Init();
WitInit(WIT_PROTOCOL_I2C, 0x50);
```

循环读取并打印 IMU 模块的数据。

```
while (1)
{
  WitReadReg(AX, 12);
  delay_ms(500);
  CmdProcess();
  if (s_cDataUpdate)
  {
    for (i = 0; i < 3; i++)
    {
      fAcc[i] = sReg[AX + i] / 32768.0f * 16.0f;
      fGyro[i] = sReg[GX + i] / 32768.0f * 2000.0f;
      fAngle[i] = sReg[Roll + i] / 32768.0f * 180.0f;
    }
    if (s_cDataUpdate & ACC_UPDATE)
    {
      printf("acc:%.3f %.3f %.3f\r\n", fAcc[0], fAcc[1], fAcc[2]);
      s_cDataUpdate &= ~ACC_UPDATE;
    }
    if (s_cDataUpdate & GYRO_UPDATE)
    {
      printf("gyro:%.3f %.3f %.3f\r\n", fGyro[0], fGyro[1], fGyro[2]);
      s_cDataUpdate &= ~GYRO_UPDATE;
    }
    if (s_cDataUpdate & ANGLE_UPDATE)
    {
      printf("angle:%.3f %.3f %.3f\r\n", fAngle[0], fAngle[1], fAngle[2]);
      s_cDataUpdate &= ~ANGLE_UPDATE;
    }
    if (s_cDataUpdate & MAG_UPDATE)
    {
      printf("mag:%d %d %d\r\n", sReg[HX], sReg[HY], sReg[HZ]);
      s_cDataUpdate &= ~MAG_UPDATE;
    }
  }
 }
}
```

3.3 实验效果

程序下载进入 STM32 后，打开猫猫串口助手（配置参数如下图所示），可以看到一直打印 IMU 模块的数据，当我们改变 IMU 模块的姿态，数据会发生变化。



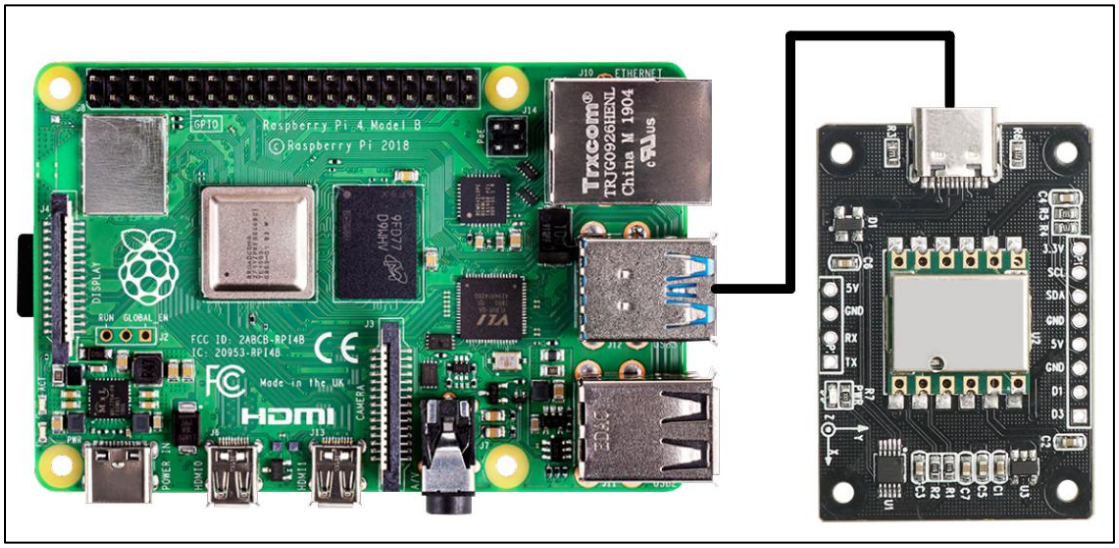
4. 树莓派-USB 应用例程

需要准备的产品：Win10 或 Win11 电脑、IMU 模块、Type-C 数据线、树莓派开发套件。

4.1 准备工作

4.1.1 硬件连接

使用 Type-C 数据线将 IMU 模块和树莓派的 USB 口连接起来。



4.1.2 安装串口驱动库

如果 Python3 里没有安装 serial 串口驱动库，请运行以下命令安装。“pip3 install pyserial”

```
ubuntu@ubuntu:~/Desktop$ pip3 install pyserial
```

输入指令可以查看“pip3 list | grep pyserial”可以查看 serial 串口驱动库是否安装，安装会列出 pyserial 及其版本号：

```
ubuntu@ubuntu:~/Desktop$ pip3 list | grep pyserial
pyserial                  3.5
```

4.2 案例开发

具体代码请看资料中的源码。

如果连接的 USB 设备不止一个，请查看并确认 IMU 模块的 USB 设备号，并修改 port 的值。程序中已经设置 USB 的波特率 baud=9600，根据 IMU 模块的波特率，修改 baud 值。

```
143 if __name__ == '__main__':
144     port = '/dev/ttyUSB0' # USB serial port
145     baud = 9600   # Same baud rate as the INERTIAL navigation module
146     ser = serial.Serial(port, baud, timeout=0.5)
147     print("Serial is Opened:", ser.is_open)
148     while(1):
149         datahex = ser.read(33)
150         DueData(datahex)
151
```

输入指令“ls /dev/ttyUSB*”，插拔 USB 数据线，判断串行端口号，可以判断出端口号为“ttyUSB0”，如果端口号和程序“imu_usb.py”中的不一致，要对应更改为 ttyUSB0

的符号

插入 USB 发现串口号为“ttyUSB0”，

```
pi@raspberrypi:~ $ ls /dev/ttyUSB*
/dev/ttyUSB0
pi@raspberrypi:~ $
```

拔出则无串口号显示，

```
pi@raspberrypi:~ $ ls /dev/ttyUSB*
ls: 无法访问'/dev/ttyUSB*': 没有那个文件或目录
pi@raspberrypi:~ $
```

```
144     port = '/dev/ttyUSB0' # USB serial port
```

4.3 实验效果

将 imu_usb.py 文件导入到系统中，打开终端，进入对应文件夹，然后运行以下命令：“python3 imu_usb.py”可以看到终端一直打印 IMU 模块的数据，当改变 IMU 模块的姿态，数据会跟着变化。

```
ubuntu@ubuntu:~/Desktop$ python3 imu_usb.py
Serial is Opened: True
acc:     -0.017      0.045      1.007
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.017      0.046      1.008
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.017      0.046      1.007
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.018      0.046      1.006
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.018      0.046      1.007
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.016      0.046      1.007
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.019      0.047      1.005
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.016      0.046      1.007
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.018      0.046      1.007
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
acc:     -0.018      0.046      1.006
gyro:      0.000       0.000       0.000
angle:     2.043      -0.049       8.399
```

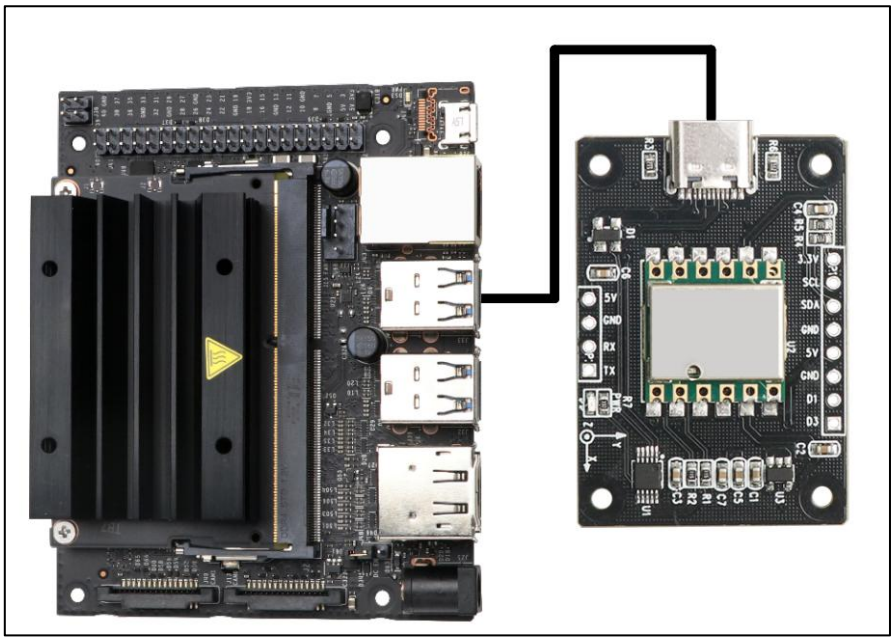
5. Jetson Nano-USB 应用例程

需要准备的产品：Win10 电脑、惯导模块、Type-C 数据线、Jetson Nano 开发套件。

5.1 准备工作

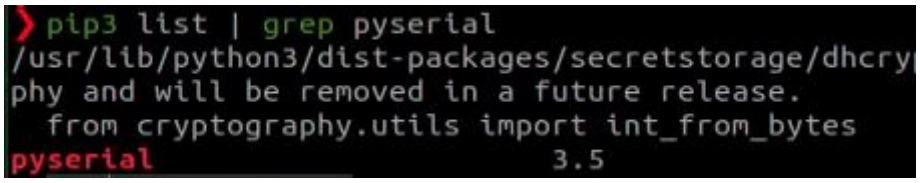
5.1.1 准备硬件连接

使用 Type-C 数据线将惯导模块和 Jetson Nano 的 USB 口连接起来。



5.1.2 安装串口驱动库

输入指令可以查看“pip3 list | grep pyserial”可以查看 serial 串口驱动库是否安装，安装会列出 pyserial 及其版本号：



如果 Python3 里没有安装 serial 串口驱动库，请运行以下命令安装。“pip3 install pyserial”



5.2 案例开发

具体代码请看资料中的源码。

如果连接的 USB 设备不止一个，请查看并确认惯导模块的 USB 设备号，并修改 port 的值。程序中已经设置 USB 的波特率 baud=9600，根据惯导模块的波特率，修改 baud 的值。

```
143  if __name__ == '__main__':
144      port = '/dev/ttyUSB1' # USB serial port
145      baud = 9600   # Same baud rate as the INERTIAL navigation module
146      ser = serial.Serial(port, baud, timeout=0.5)
147      print("Serial is Opened:", ser.is_open)
148      while(1):
149          datahex = ser.read(33)
150          DueData(datahex)
```

输入指令“ll /dev/ttyUSB*”，插拔 USB 数据线，判断串行端口号，可以判断出端口号为“ttyUSB1”

```
❯ ll /dev/ttyUSB*
crwxrwxrwx 1 root dialout 188, 0 Sep 27 10:24 /dev/ttyUSB0
```

```
❯ ll /dev/ttyUSB*
crwxrwxrwx 1 root dialout 188, 0 Sep 27 10:24 /dev/ttyUSB0
crwxrwxrwx 1 root dialout 188, 1 Sep 27 10:28 /dev/ttyUSB1
```

在上述程序“imu_usb.py”中对应更改 ttyUSB1 的符号

```
144      port = '/dev/ttyUSB1' # USB serial port
```

5.3 实验效果

将 imu_usb.py 文件导入到系统中，打开终端，进入对应文件夹，然后运行以下命令：“python3 imu_usb.py”可以看到终端一直打印惯导模块的数据，当改变惯导模块的姿态，数据会跟着变化。

```
❯ python3 imu_usb.py
Serial is Opened: True
acc:     0.133      0.185      0.980
gyro:     0.000      0.000      0.000
angle:   10.058     -8.487    168.970
acc:     0.132      0.184      0.979
gyro:     0.000      0.000      0.000
angle:   10.052     -8.487    168.964
acc:     0.133      0.184      0.980
gyro:     0.000      0.000      0.000
angle:   10.052     -8.492    168.964
acc:     0.133      0.184      0.980
gyro:     0.000      0.000      0.000
angle:   10.052     -8.492    168.959
acc:     0.132      0.185      0.979
gyro:     0.000      0.000      0.000
angle:   10.047     -8.498    168.959
acc:     0.132      0.184      0.980
gyro:     0.000      0.000      0.000
angle:   10.047     -8.498    168.953
acc:     0.133      0.184      0.980
gyro:     0.000      0.000      0.000
angle:   10.047     -8.498    168.953
acc:     0.132      0.185      0.980
gyro:     0.000      0.000      0.000
angle:   10.047     -8.503    168.948
acc:     0.132      0.184      0.979
gyro:     0.000      0.000      0.000
angle:   10.042     -8.503    168.948
acc:     0.133      0.185      0.980
gyro:     0.000      0.000      0.000
angle:   10.042     -8.503    168.948
acc:     0.133      0.184      0.979
gyro:     0.000      0.000      0.000
angle:   10.042     -8.509    168.942
acc:     0.132      0.184      0.980
gyro:     0.000      0.000      0.000
angle:   10.042     -8.509    168.942
acc:     0.133      0.185      0.980
gyro:     0.000      0.000      0.000
angle:   10.036     -8.509    168.942
acc:     0.132      0.184      0.979
gyro:     0.000      0.000      0.000
angle:   10.036     -8.509    168.937
```

6. Arduino-串口应用例程

本次例程使用的是 Arduino UNO 开发板，为了演示方便以下内容简称为 Arduino 。

需要准备的产品：Win10/Win11 电脑、IMU 模块（数据接口已引线）、杜邦线若干。

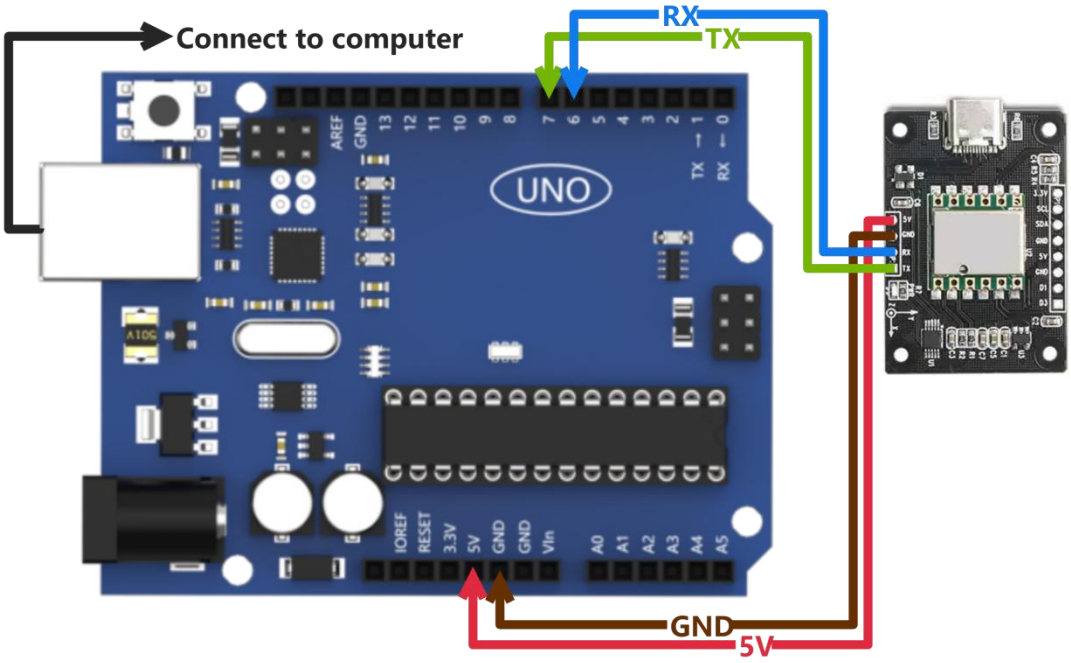
6.1 准备工作

将 Arduino 的烧录线连接到电脑。将 Arduino 串口连接到 IMU 模块的串口接口。

接线说明：

Arduino		IMU
5V	------>	5V
GND	------>	GND
IO7	<------	TXD

IO6 <------ RXD



6.2 案例开发

具体代码请看资料中的源码。

本例程将串口打印的波特率设置为115200 bps,将IMU模块连接的波特率设置为9600。可以根据 IMU 模块的波特率修改 Arduino 串口的波特率。

```
IMU_Serial.begin(9600);
Serial.begin(115200);
```

读取并打印 IMU 模块的数据。

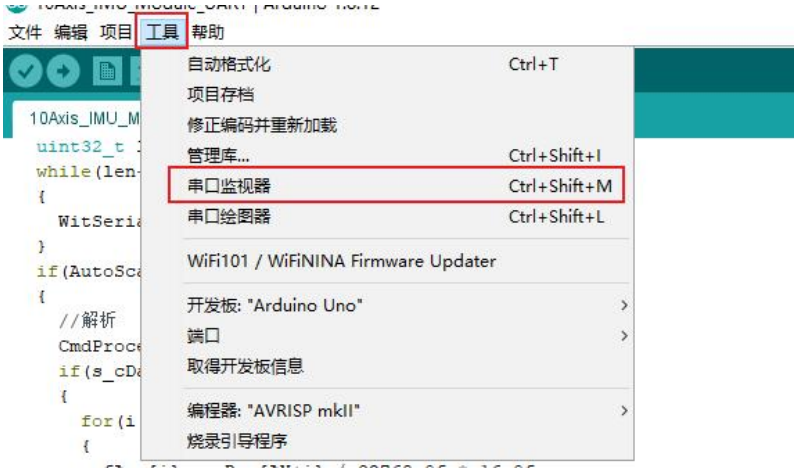
```
while(len--)
{
  WitSerialDataIn((char)IMU_Serial.read());
}
if(AutoScanSensor())
{
  //解析
  CmdProcess();
  if(s_cDataUpdate)
  {
    for(i = 0; i < 3; i++)
    {
      fAcc[i] = sReg[AX+i] / 32768.0f * 16.0f;
      fGyro[i] = sReg[GX+i] / 32768.0f * 2000.0f;
    }
    if(sReg[TEMP] - sReg[Roll] > 10 | sReg[Roll] - sReg[TEMP] > 10)
    {
      fAngle[0] = sReg[Roll] / 32768.0f * 180.0f;
      fAngle_tmp = fAngle[0];
    }
    else   fAngle[0] = fAngle_tmp;

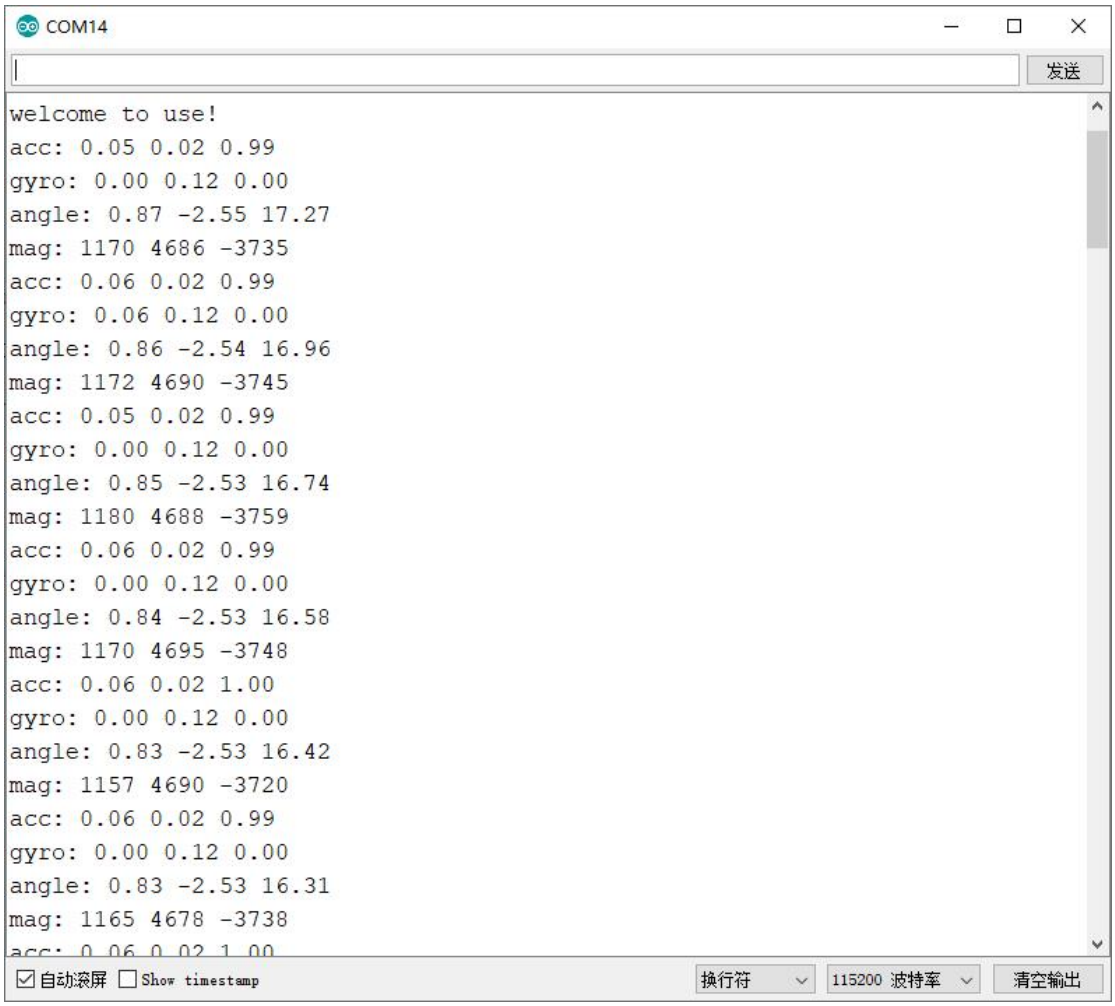
    for(i = 1; i<3;i++)
    {
      fAngle[i] = sReg[Roll+i] / 32768.0f * 180.0f;
    }

    if(s_cDataUpdate & ACC_UPDATE)
    {
      Serial.print("acc: ");
      Serial.print(fAcc[0]);
      Serial.print(" ");
      Serial.print(fAcc[1]);
      Serial.print(" ");
      Serial.println(fAcc[2]);
      s_cDataUpdate &= ~ACC_UPDATE;
    }
    if(s_cDataUpdate & GYRO_UPDATE)
    {
      Serial.print("gyro: ");
      Serial.print(fGyro[0]);
      Serial.print(" ");
      Serial.print(fGyro[1]);
      Serial.print(" ");
```

6.3 实验效果

程序下载进入 Arduino 后，打开工具栏目的串口监视器（如下图所示），波特率为 115200，可以看到一直打印 IMU 模块的数据，当我们改变 IMU 模块的姿态，数据会发生变化。





7. Arduino-IIC 应用例程

本次例程使用的是 Arduino UNO 开发板，为了演示方便以下内容简称为 Arduino 。

需要准备的产品：Win10/Win11 电脑、IMU 模块（数据接口已引线）、type-c 数据线、杜邦线若干。

7.1 准备工作

将 Arduino 的烧录线连接到电脑。将 Arduino 与 IMU 模块的 I2C 接口。

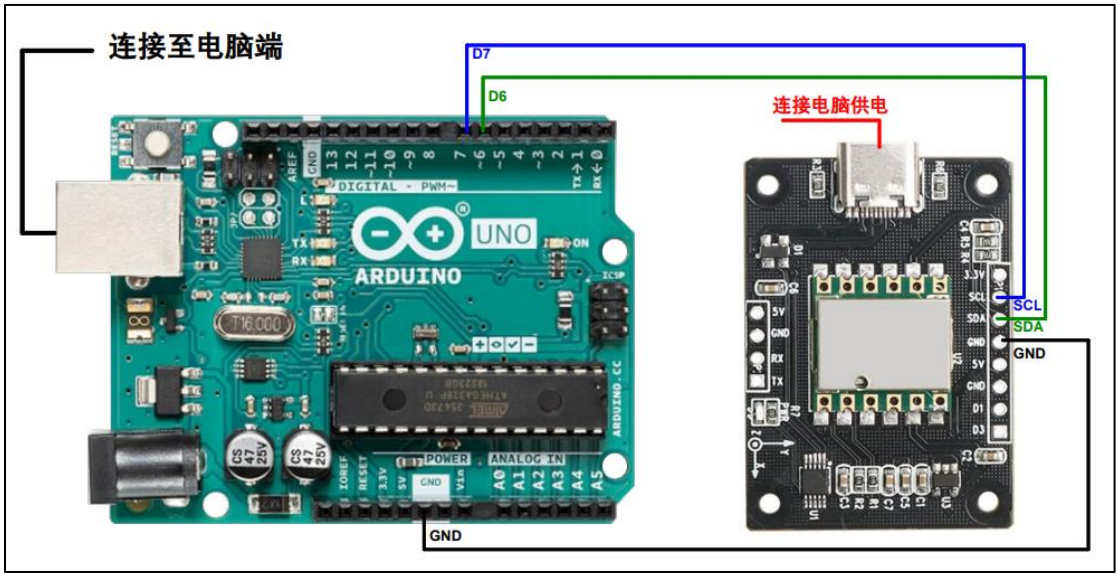
接线说明：

Arduino IMU

GND ------> GND

D6 ------> SDA

D7 ------> SCL



7.2 案例开发

具体代码请看资料中的源码。

本例本例程已经将波特率设置为 115200 bps，注意此波特率是 Arduino 串口打印的波特率，并非 IMU 模块的波特率。

```
Serial.begin(115200);
```

接下来需要注意的是 IMU 模块的 IIC 地址，默认为 0x50，如果修改了其他的地址请修改此参数。

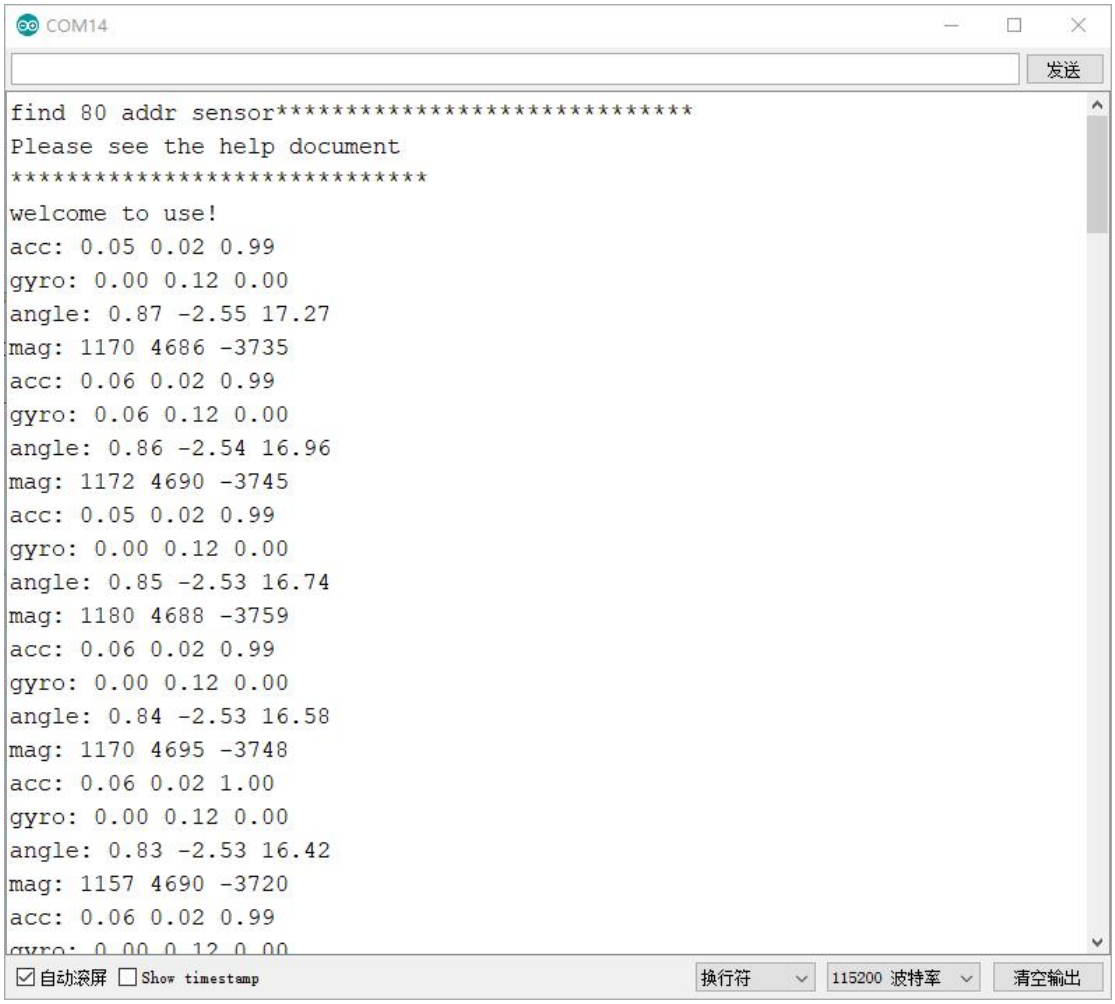
```
IIC_Init();
WitInit(WIT_PROTOCOL_I2C, 0x50);
```

循环读取并打印 IMU 模块的数据。

```
void loop() {
  // put your main code here, to run repeatedly:
  float fAcc[3], fGyro[3], fAngle[3];
  int i;
  WitReadReg(AX, 12);
  delay(500);
  CmdProcess();
  if (s_cDataUpdate)
  {
    for (i = 0; i < 3; i++)
    {
      fAcc[i] = sReg[AX + i] / 32768.0f * 16.0f;
      fGyro[i] = sReg[GX + i] / 32768.0f * 2000.0f;
      fAngle[i] = sReg[Roll + i] / 32768.0f * 180.0f;
    }
    if (s_cDataUpdate & ACC_UPDATE)
    {
      Serial.print("acc: ");
      Serial.print(fAcc[0]);
      Serial.print(" ");
      Serial.print(fAcc[1]);
      Serial.print(" ");
      Serial.println(fAcc[2]);
      s_cDataUpdate &= ~ACC_UPDATE;
    }
    if (s_cDataUpdate & GYRO_UPDATE)
    {
      Serial.print("gyro: ");
```

7.3 实验效果

程序下载进入 Arduino 后，打开工具栏目的串口监视器（如下图所示），波特率为 115200，可以看到一直打印 IMU 模块的数据，当我们改变 IMU 模块的姿态，数据会发生变化。


COM14
发送
find 80 addr sensor*******************************
Please see the help document
****
welcome to use!
acc: 0.05 0.02 0.99
gyro: 0.00 0.12 0.00
angle: 0.87 -2.55 17.27
mag: 1170 4686 -3735
acc: 0.06 0.02 0.99
gyro: 0.06 0.12 0.00
angle: 0.86 -2.54 16.96
mag: 1172 4690 -3745
acc: 0.05 0.02 0.99
gyro: 0.00 0.12 0.00
angle: 0.85 -2.53 16.74
mag: 1180 4688 -3759
acc: 0.06 0.02 0.99
gyro: 0.00 0.12 0.00
angle: 0.84 -2.53 16.58
mag: 1170 4695 -3748
acc: 0.06 0.02 1.00
gyro: 0.00 0.12 0.00
angle: 0.83 -2.53 16.42
mag: 1157 4690 -3720
acc: 0.06 0.02 0.99
自动滚屏
Show timestamp
换行符
115200 波特率
清空输出